VibeMeGood — Complete Build Specification

The Full-Stack Decision Intelligence Terminal for PrizePicks

Version: Final
Hand this entire document to Replit Agent. Attach VibeMeGood-main.zip as the existing codebase.

WHAT THIS IS

This is not a sports picks app. It is a **decision intelligence terminal** that operates inside the PrizePicks market. The edge comes from one thing: most PrizePicks users are emotional and impulsive. This platform makes one user systematic and process-driven.

Every feature exists to either find an edge, quantify it precisely, or protect the user from destroying it with bad behavior. If a feature doesn't do one of those three things, cut it.

EXISTING CODEBASE CONTEXT

The attached zip (VibeMeGood-main.zip) is a React + Express monorepo with this structure:

```
artifacts/prizepicks/          React frontend (Wouter, Tailwind, shadcn/ui, TanSt
artifacts/api-server/          Express backend (TypeScript)
artifacts/api-server/src/routes/   All API routes — most are working CRUD
artifacts/api-server/src/lib/cron.ts   Cron jobs — ALL STUBS, need real implement
artifacts/api-server/src/routes/sync.ts   Sync triggers — ALL STUBS
lib/db/                        Drizzle ORM schema + PostgreSQL
lib/integrations-anthropic-ai/ Anthropic SDK wrapper
```

What already works:

- All basic CRUD routes (players, teams, games, pp-lines, entries, injuries, etc.)
- Clerk authentication
- Stripe billing integration
- Anthropic AI chat (conversations, messages, streaming explain)

- Settings page with individual sync trigger buttons
- Dashboard, Slate Board, Entry Builder, Journal, Review, AI Chat pages (UI only — no real data)

What is stubbed and needs real implementation:

- `syncPpLinesImpl()` — returns `{ recordsProcessed: 0 }`
- `syncExternalOddsImpl()` — returns `{ recordsProcessed: 0 }`
- `syncInjuriesImpl()` — returns `{ recordsProcessed: 0 }`
- `syncProjectionsImpl()` — returns `{ recordsProcessed: 0 }`
- All prop score calculation
- All edge scoring

Your primary job: Implement every stub with real logic, add the new tables and routes below, and connect every frontend page to real backend data.

TECH STACK — DO NOT CHANGE

- **Frontend:** React, Wouter, Tailwind CSS, shadcn/ui, TanStack Query
- **Backend:** Express.js, TypeScript
- **Database:** PostgreSQL, Drizzle ORM
- **Auth:** Clerk
- **AI:** Anthropic Claude API (server-side only — never expose key to frontend)
- **Scheduler:** node-cron
- **Real-time:** Server-Sent Events (SSE) — simpler than WebSocket, works perfectly in Express
- **Hosting:** Replit

ENVIRONMENT VARIABLES

Add all to Replit Secrets:

```
# Already exists
DATABASE_URL=postgresql://...
CLERK_SECRET_KEY=sk_...
CLERK_PUBLISHABLE_KEY=pk_...
STRIPE_SECRET_KEY=sk_...
ANTHROPIC_API_KEY=sk-ant-...

# Add these
ODDS_API_KEY=                    # the-odds-api.com — free tier, 500 req/month
PP_API_BASE=https://api.prizepicks.com
ODDS_API_BASE=https://api.the-odds-api.com/v4
NBA_STATS_BASE=https://stats.nba.com/stats
NHL_API_BASE=https://api.nhle.com/stats/rest/en
BB_SAVANT_BASE=https://baseballsavant.mlb.com
```

PART 1: DATABASE SCHEMA ADDITIONS

Add these tables to `lib/db/schema.ts`. Do not modify existing tables — only add.

```
// ─── PLAYER ID MAPPING ───
// Links PrizePicks player names to external API numeric IDs
export const playerIdMappingTable = pgTable("player_id_mapping", {
  id: serial("id").primaryKey(),
  playerId: integer("player_id").references(() => playersTable.id),
  source: varchar("source", { length: 50 }).notNull(),   // nba | nhl | mlb | nfl
  externalId: varchar("external_id", { length: 100 }).notNull(),
  externalName: varchar("external_name", { length: 200 }),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

// ─── OUR PROJECTION MODEL OUTPUT ───
// Stores projections generated by our own weighted formula
export const ourProjectionsTable = pgTable("our_projections", {
  id: serial("id").primaryKey(),
  playerId: integer("player_id").references(() => playersTable.id),
  gameId: integer("game_id").references(() => gamesTable.id),
  statType: varchar("stat_type", { length: 100 }).notNull(),
  projectedValue: numeric("projected_value").notNull(),
  weightedAvg: numeric("weighted_avg"),           // raw L10 weighted average
  paceFactor: numeric("pace_factor"),             // opponent pace adjustment
  defenseFactor: numeric("defense_factor"),        // opponent defense adjustment
  restFactor: numeric("rest_factor"),             // back-to-back penalty
```

```
  gamesUsed: integer("games_used"),               // how many games in the sample
  confidence: varchar("confidence", { length: 20 }), // high | medium | low
  modelVersion: varchar("model_version", { length: 20 }).default("v1"),
  generatedAt: timestamp("generated_at").defaultNow(),
});

// ─── PROBABILITY CALIBRATION ───
// Self-updates after every logged pick result
export const probabilityCalibrationTable = pgTable("probability_calibration", {
  id: serial("id").primaryKey(),
  sport: varchar("sport", { length: 50 }).notNull(),
  statType: varchar("stat_type", { length: 100 }).notNull(),
  lineType: varchar("line_type", { length: 20 }).notNull(),   // goblin | demon |
  edgeBucket: varchar("edge_bucket", { length: 20 }).notNull(), // high | mid | l
  direction: varchar("direction", { length: 10 }).notNull(),    // more | less
  sampleSize: integer("sample_size").default(0),
  hitCount: integer("hit_count").default(0),
  hitRate: numeric("hit_rate"),
  confidenceInterval: numeric("confidence_interval"),  // 95% CI half-width
  lastUpdated: timestamp("last_updated").defaultNow(),
}, (t) => ({
  uniq: uniqueIndex("prob_cal_unique").on(t.sport, t.statType, t.lineType, t.edge
}));

// ─── CLOSING LINE VALUE ───
export const clvRecordsTable = pgTable("clv_records", {
  id: serial("id").primaryKey(),
  entryPickId: integer("entry_pick_id").references(() => entryPicksTable.id),
  ppLineId: integer("pp_line_id").references(() => ppLinesTable.id),
  lockedLine: numeric("locked_line").notNull(),
  closingLine: numeric("closing_line"),
  clv: numeric("clv"),                            // closing_line - locked_line (
  direction: varchar("direction", { length: 10 }),
  createdAt: timestamp("created_at").defaultNow(),
});

// ─── PLAYER STREAKS ───
export const playerStreaksTable = pgTable("player_streaks", {
  id: serial("id").primaryKey(),
  playerId: integer("player_id").references(() => playersTable.id),
  statType: varchar("stat_type", { length: 100 }).notNull(),
  currentStreak: integer("current_streak").default(0), // positive=over streak, n
  streakType: varchar("streak_type", { length: 10 }),  // over | under
  updatedAt: timestamp("updated_at").defaultNow(),
}, (t) => ({
  uniq: uniqueIndex("streak_unique").on(t.playerId, t.statType),
}));
```

```
// ─── GAME ENVIRONMENT ───
export const gameEnvironmentTable = pgTable("game_environment", {
  id: serial("id").primaryKey(),
  gameId: integer("game_id").references(() => gamesTable.id).unique(),
  gameTotal: numeric("game_total"),
  impliedPace: numeric("implied_pace"),
  environmentScore: integer("environment_score"),  // 0-100
  notes: text("notes"),
  updatedAt: timestamp("updated_at").defaultNow(),
});

// ─── HEAD-TO-HEAD MATCHUP HISTORY ───
export const matchupHistoryTable = pgTable("matchup_history", {
  id: serial("id").primaryKey(),
  playerId: integer("player_id").references(() => playersTable.id),
  opponentTeamId: integer("opponent_team_id").references(() => teamsTable.id),
  statType: varchar("stat_type", { length: 100 }).notNull(),
  gamesPlayed: integer("games_played").default(0),
  avgValue: numeric("avg_value"),
  overRateAtCurrentLine: numeric("over_rate_at_current_line"),
  updatedAt: timestamp("updated_at").defaultNow(),
}, (t) => ({
  uniq: uniqueIndex("matchup_unique").on(t.playerId, t.opponentTeamId, t.statType
}));

// ─── USER SETTINGS (PERSISTED PER CLERK USER) ───
export const userSettingsTable = pgTable("user_settings", {
  id: serial("id").primaryKey(),
  userId: varchar("user_id", { length: 255 }).notNull().unique(),
  bankroll: numeric("bankroll").default("500"),
  unitSize: numeric("unit_size").default("25"),
  kellyFraction: numeric("kelly_fraction").default("0.25"),
  maxUnitsPerEntry: integer("max_units_per_entry").default(1),
  dailyLossLimit: numeric("daily_loss_limit"),
  hitRateAssumptions: jsonb("hit_rate_assumptions").default({
    goblinOver: 0.65, standardHigh: 0.58, standardMid: 0.52,
    standardLow: 0.47, demonUnder: 0.65, demonOver: 0.35
  }),
  payoutConfig: jsonb("payout_config").default({
    power: { 2:3, 3:6, 4:10, 5:20, 6:40 }
  }),
  edgeWeights: jsonb("edge_weights").default({ marketGap:60, lineType:25, ourProj
  excludeDemonOvers: boolean("exclude_demon_overs").default(true),
  minEdgeToPlay: numeric("min_edge_to_play").default("4"),
  aiModel: varchar("ai_model", { length: 100 }).default("claude-sonnet-4-20250514
  excludedSports: jsonb("excluded_sports").default([]),
```

```
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

// ─── SESSION TRACKING (LOSS LIMITS) ───
export const sessionLogsTable = pgTable("session_logs", {
  id: serial("id").primaryKey(),
  userId: varchar("user_id", { length: 255 }).notNull(),
  sessionDate: date("session_date").notNull(),
  totalStaked: numeric("total_staked").default("0"),
  totalPnl: numeric("total_pnl").default("0"),
  entriesPlaced: integer("entries_placed").default(0),
  isLocked: boolean("is_locked").default(false),
  createdAt: timestamp("created_at").defaultNow(),
}, (t) => ({
  uniq: uniqueIndex("session_unique").on(t.userId, t.sessionDate),
}));

// ─── BEHAVIORAL LOGS ───
export const behavioralLogsTable = pgTable("behavioral_logs", {
  id: serial("id").primaryKey(),
  userId: varchar("user_id", { length: 255 }).notNull(),
  entryId: integer("entry_id").references(() => entriesTable.id),
  timeOfDay: varchar("time_of_day", { length: 20 }),
  minutesSinceLastLoss: integer("minutes_since_last_loss"),
  deviatedFromOptimizer: boolean("deviated_from_optimizer").default(false),
  picksChangedFromOptimizer: integer("picks_changed_from_optimizer").default(0),
  stakeMultipleOfUnit: numeric("stake_multiple_of_unit"),
  emotionalState: varchar("emotional_state", { length: 50 }),
  secondsToDecision: integer("seconds_to_decision"),
  createdAt: timestamp("created_at").defaultNow(),
});

// ─── LINE MOVE EVENTS ───
export const lineMoveEventsTable = pgTable("line_move_events", {
  id: serial("id").primaryKey(),
  ppLineId: integer("pp_line_id").references(() => ppLinesTable.id),
  bookName: varchar("book_name", { length: 100 }),
  prevLine: numeric("prev_line"),
  newLine: numeric("new_line"),
  moveSize: numeric("move_size"),
  moveDirection: varchar("move_direction", { length: 10 }),
  sequenceNumber: integer("sequence_number"),
  capturedAt: timestamp("captured_at").defaultNow(),
});
```

PART 2: BACKEND IMPLEMENTATIONS

2A. PrizePicks Sync

File: `artifacts/api-server/src/lib/sync/prizepicks.ts`

```
import { db } from "@workspace/db";
import { ppLinesTable, ppLineHistoryTable, playersTable, teamsTable } from "@work
import { eq, and } from "drizzle-orm";

const PP_BASE = process.env.PP_API_BASE || "https://api.prizepicks.com";

export async function syncPpLines(): Promise<number> {
  const res = await fetch(
    `${PP_BASE}/projections?per_page=500&single_stat=true&include=new_player,leag
    { headers: { "Accept": "application/json" } }
  );
  if (!res.ok) throw new Error(`PrizePicks API error: ${res.status}`);
  const data = await res.json();

  const playerMap: Record<string, any> = {};
  const leagueMap: Record<string, any> = {};
  for (const inc of (data.included || [])) {
    if (inc.type === "new_player") playerMap[inc.id] = inc.attributes;
    if (inc.type === "league") leagueMap[inc.id] = inc.attributes;
  }

  let processed = 0;
  const seenLineIds = new Set<number>();

  for (const proj of (data.data || [])) {
    try {
      const pAttr = playerMap[proj.relationships?.new_player?.data?.id] || {};
      const lAttr = leagueMap[proj.relationships?.league?.data?.id] || {};
      const lineValue = parseFloat(proj.attributes.line_score);
      const lineType = (proj.attributes.line_type || "standard").toLowerCase();
      const statType = proj.attributes.stat_type;
      const sport = lAttr.name || pAttr.sport || "Unknown";
      const playerName = pAttr.name || "Unknown";
      const teamAbbr = pAttr.team || "";

      // Upsert player
      let [player] = await db.select().from(playersTable)
        .where(and(eq(playersTable.fullName, playerName), eq(playersTable.sport,
        .limit(1);
```

```
if (!player) {
  [player] = await db.insert(playersTable).values({
    sport, fullName: playerName,
    firstName: playerName.split(" ")[0] || "",
    lastName: playerName.split(" ").slice(1).join(" ") || "",
    team: teamAbbr, status: "active",
    externalIds: { pp_id: proj.relationships?.new_player?.data?.id },
  }).returning();
} else if (player.team !== teamAbbr && teamAbbr) {
  await db.update(playersTable).set({ team: teamAbbr, updatedAt: new Date()
    .where(eq(playersTable.id, player.id));
}

// Check for existing active line
const [existing] = await db.select().from(ppLinesTable)
  .where(and(
    eq(ppLinesTable.playerId, player.id),
    eq(ppLinesTable.statType, statType),
    eq(ppLinesTable.isActive, true)
  )).limit(1);

let lineId: number;
if (existing) {
  seenLineIds.add(existing.id);
  lineId = existing.id;
  // Record movement if line or type changed
  if (Number(existing.lineValue) !== lineValue || existing.lineType !== lin
    await db.insert(ppLineHistoryTable).values({
      ppLineId: existing.id,
      lineValue: Number(existing.lineValue).toString(),
      lineType: existing.lineType,
      capturedAt: new Date(),
    });
    await db.update(ppLinesTable).set({
      lineValue: lineValue.toString(), lineType, updatedAt: new Date()
    }).where(eq(ppLinesTable.id, existing.id));
  }
} else {
  const [newLine] = await db.insert(ppLinesTable).values({
    playerId: player.id, statType,
    lineValue: lineValue.toString(), lineType,
    directionalityType: "over_under", isActive: true,
    openedAt: new Date(),
  }).returning();
  await db.insert(ppLineHistoryTable).values({
    ppLineId: newLine.id, lineValue: lineValue.toString(),
    lineType, capturedAt: new Date(),
```

```
        });
        seenLineIds.add(newLine.id);
        lineId = newLine.id;
      }
      processed++;
    } catch (e) {
      console.error("Error processing PP projection:", e);
    }
  }

  return processed;
}
```

2B. The Odds API Sync

File: `artifacts/api-server/src/lib/sync/external-odds.ts`

```
import { db } from "@workspace/db";
import { externalLinesTable, ppLinesTable, playersTable, propScoresTable, lineMov
import { eq, and } from "drizzle-orm";

const ODDS_BASE = process.env.ODDS_API_BASE || "https://api.the-odds-api.com/v4";
const ODDS_KEY = process.env.ODDS_API_KEY || "";

const SPORT_KEYS: Record<string, string> = {
  "NBA": "basketball_nba", "MLB": "baseball_mlb",
  "NHL": "icehockey_nhl", "NFL": "americanfootball_nfl",
  "WNBA": "basketball_wnba",
};

const STAT_MARKETS: Record<string, string> = {
  "Points": "player_points", "Rebounds": "player_rebounds",
  "Assists": "player_assists", "3-Pointers Made": "player_threes",
  "Pts+Reb+Ast": "player_points_rebounds_assists",
  "Total Bases": "player_total_bases", "Hits": "player_hits",
  "Strikeouts": "pitcher_strikeouts",
};

const BOOKS = ["draftkings", "fanduel", "espnbet"];

export async function syncExternalOdds(): Promise<number> {
  if (!ODDS_KEY) { console.warn("ODDS_API_KEY not set — skipping external odds sy

  const activeLines = await db
    .select({ line: ppLinesTable, player: playersTable })
    .from(ppLinesTable)
```

```
  .innerJoin(playersTable, eq(ppLinesTable.playerId, playersTable.id))
  .where(eq(ppLinesTable.isActive, true));

const bySport: Record<string, typeof activeLines> = {};
for (const r of activeLines) {
  const s = r.player.sport;
  if (!bySport[s]) bySport[s] = [];
  bySport[s].push(r);
}

let processed = 0;
for (const [sport, lines] of Object.entries(bySport)) {
  const sportKey = SPORT_KEYS[sport];
  if (!sportKey) continue;

  try {
    const eventsRes = await fetch(`${ODDS_BASE}/sports/${sportKey}/events?apiKe
    if (!eventsRes.ok) continue;
    const events = await eventsRes.json();

    const neededMarkets = new Set(
      lines.map(l => STAT_MARKETS[l.line.statType]).filter(Boolean)
    );
    if (!neededMarkets.size) continue;

    for (const event of events.slice(0, 15)) {
      const oddsRes = await fetch(
        `${ODDS_BASE}/sports/${sportKey}/events/${event.id}/odds?` +
        `apiKey=${ODDS_KEY}&regions=us&markets=${[...neededMarkets].join(",")}&
      );
      if (!oddsRes.ok) continue;
      const oddsData = await oddsRes.json();

      for (const bookmaker of (oddsData.bookmakers || [])) {
        for (const market of (bookmaker.markets || [])) {
          for (const outcome of (market.outcomes || [])) {
            if (outcome.type !== "Over") continue;
            const playerName = outcome.description || outcome.name;
            if (!playerName || !outcome.point) continue;

            const match = lines.find(l => {
              const ppLast = l.player.fullName.split(" ").pop()?.toLowerCase()
              const mktLast = playerName.split(" ").pop()?.toLowerCase() || "";
              return ppLast === mktLast || l.player.fullName.toLowerCase() ===
            });
            if (!match) continue;
```

```
              const newVal = outcome.point.toString();
              const [existing] = await db.select().from(externalLinesTable)
                .where(and(
                  eq(externalLinesTable.ppLineId, match.line.id),
                  eq(externalLinesTable.bookName, bookmaker.key)
                )).limit(1);

              if (existing && existing.lineValue !== newVal) {
                await db.insert(lineMoveEventsTable).values({
                  ppLineId: match.line.id,
                  bookName: bookmaker.key,
                  prevLine: existing.lineValue,
                  newLine: newVal,
                  moveSize: (parseFloat(newVal) - parseFloat(existing.lineValue))
                  moveDirection: parseFloat(newVal) > parseFloat(existing.lineVal
                  capturedAt: new Date(),
                });
              }

              await db.insert(externalLinesTable).values({
                playerId: match.player.id,
                ppLineId: match.line.id,
                bookName: bookmaker.key,
                statType: match.line.statType,
                lineValue: newVal,
                capturedAt: new Date(),
              }).onConflictDoUpdate({
                target: [externalLinesTable.ppLineId, externalLinesTable.bookName
                set: { lineValue: newVal, capturedAt: new Date() }
              });
              processed++;
            }
          }
        }
        await new Promise(r => setTimeout(r, 150)); // rate limit
      }
    } catch (e) { console.error(`External odds sync error ${sport}:`, e); }
  }

  await recalcPropScores();
  return processed;
}

async function recalcPropScores() {
  const lines = await db.select({ line: ppLinesTable, player: playersTable })
    .from(ppLinesTable)
    .innerJoin(playersTable, eq(ppLinesTable.playerId, playersTable.id))
```

```
    .where(eq(ppLinesTable.isActive, true));

  for (const { line, player } of lines) {
    try {
      const extLines = await db.select().from(externalLinesTable)
        .where(eq(externalLinesTable.ppLineId, line.id));

      let edgeScore = 0;
      let marketSupportScore = 50;

      if (extLines.length >= 2) {
        const vals = extLines.map(l => parseFloat(l.lineValue.toString()));
        const marketAvg = vals.reduce((a, b) => a + b, 0) / vals.length;
        const ppLine = parseFloat(line.lineValue.toString());
        const gap = ppLine - marketAvg;
        const trueEdge = (-gap / marketAvg) * 100;
        edgeScore = trueEdge;
        marketSupportScore = Math.max(0, Math.min(100, 50 + trueEdge * 3));
      }

      const lineBonus = line.lineType === "goblin" ? 12 : line.lineType === "demo
      const finalScore = edgeScore + lineBonus;
      const actionTag = finalScore >= 5 ? "PLAY" : finalScore <= -4 ? "PASS" : "W

      await db.insert(propScoresTable).values({
        ppLineId: line.id, playerId: line.playerId,
        statType: line.statType,
        marketSupportScore: marketSupportScore.toString(),
        edgeScore: edgeScore.toString(),
        finalScore: finalScore.toString(),
        actionTag, scoredAt: new Date(),
      }).onConflictDoUpdate({
        target: [propScoresTable.ppLineId],
        set: {
          marketSupportScore: marketSupportScore.toString(),
          edgeScore: edgeScore.toString(),
          finalScore: finalScore.toString(),
          actionTag, scoredAt: new Date(),
        }
      });
    } catch (e) { console.error("Prop score calc error:", e); }
  }
}
```

2C. Our Own Projection Model (Free — No Paid API)

File: `artifacts/api-server/src/lib/sync/projections.ts`

This is our own statistical model built from free public APIs. No paid service needed.

How it works:

1. Pull the player's last 10 game log from the sport's free API
2. Apply a recency-weighted average (most recent games count more)
3. Adjust for opponent defensive quality
4. Adjust for pace (basketball) or starting pitcher (baseball)
5. Flag confidence based on sample size

```
import { db } from "@workspace/db";
import { ourProjectionsTable, playersTable, gamesTable, playerIdMappingTable, tea
import { eq, and } from "drizzle-orm";

// Recency weights — most recent game (index 0) gets highest weight
const WEIGHTS_10 = [0.20, 0.17, 0.14, 0.12, 0.10, 0.08, 0.07, 0.05, 0.04, 0.03];
const WEIGHTS_5  = [0.30, 0.25, 0.20, 0.15, 0.10];

// ─── NBA (stats.nba.com — no key required) ───
async function fetchNBAGameLog(nbaPlayerId: string, numGames: number) {
  const season = "2025-26";
  const url = `https://stats.nba.com/stats/playergamelog?PlayerID=${nbaPlayerId}&
  const res = await fetch(url, {
    headers: {
      "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/
      "Referer": "https://www.nba.com/",
      "Origin": "https://www.nba.com",
      "Accept": "application/json",
    }
  });
  if (!res.ok) throw new Error(`NBA Stats API error: ${res.status}`);
  const data = await res.json();
  const headers: string[] = data.resultSets[0].headers;
  const rows: any[][] = data.resultSets[0].rowSet.slice(0, numGames);
  return rows.map(row => Object.fromEntries(headers.map((h, i) => [h, row[i]])));
}

async function fetchNBATeamStats(teamId: string) {
  const url = `https://stats.nba.com/stats/teamdashboardbygeneralsplits?TeamID=${
  const res = await fetch(url, {
    headers: {
      "User-Agent": "Mozilla/5.0",
```

```
      "Referer": "https://www.nba.com/",
      "Origin": "https://www.nba.com",
    }
  });
  if (!res.ok) return null;
  const data = await res.json();
  const headers: string[] = data.resultSets[0].headers;
  const row: any[] = data.resultSets[0].rowSet[0] || [];
  return Object.fromEntries(headers.map((h, i) => [h, row[i]]));
}

const NBA_STAT_MAP: Record<string, string> = {
  "Points": "PTS", "Rebounds": "REB", "Assists": "AST",
  "3-Pointers Made": "FG3M", "Pts+Reb+Ast": "PTS", // PRA calculated separately
  "Steals": "STL", "Blocks": "BLK",
};

async function buildNBAProjection(playerId: number, statType: string, opponentExt
  const [mapping] = await db.select().from(playerIdMappingTable)
    .where(and(eq(playerIdMappingTable.playerId, playerId), eq(playerIdMappingTab
    .limit(1);

  if (!mapping) return null;

  const gameLog = await fetchNBAGameLog(mapping.externalId, 10);
  if (!gameLog.length) return null;

  const statKey = NBA_STAT_MAP[statType];
  if (!statKey) return null;

  let statValues: number[];
  if (statType === "Pts+Reb+Ast") {
    statValues = gameLog.map(g => (g.PTS || 0) + (g.REB || 0) + (g.AST || 0));
  } else {
    statValues = gameLog.map(g => parseFloat(g[statKey] || "0"));
  }

  const weights = statValues.length >= 7 ? WEIGHTS_10.slice(0, statValues.length)
  const wSum = weights.reduce((a, b) => a + b, 0);
  const normalized = weights.map(w => w / wSum);
  const weightedAvg = statValues.reduce((sum, val, i) => sum + val * (normalized[

  // Opponent defense adjustment (optional — requires opponent team ID mapping)
  let defenseFactor = 1.0;
  if (opponentExternalId) {
    try {
      const teamStats = await fetchNBATeamStats(opponentExternalId);
```

```
      if (teamStats) {
        const oppDefRtg = parseFloat(teamStats.DEF_RATING || "110");
        const leagueAvgDefRtg = 113.0; // approximate 2025-26 average
        defenseFactor = leagueAvgDefRtg / oppDefRtg;
      }
    } catch { /* non-fatal */ }
  }

  const projection = weightedAvg * defenseFactor;
  const confidence = statValues.length >= 8 ? "high" : statValues.length >= 5 ? "

  return {
    projectedValue: Math.round(projection * 10) / 10,
    weightedAvg: Math.round(weightedAvg * 10) / 10,
    defenseFactor: Math.round(defenseFactor * 1000) / 1000,
    paceFactor: 1.0,
    gamesUsed: statValues.length,
    confidence,
  };
}

// ─── NHL (api.nhle.com — no key required) ───
async function fetchNHLGameLog(nhlePlayerId: string) {
  const url = `https://api.nhle.com/stats/rest/en/player/${nhlePlayerId}/game-log
  const res = await fetch(url, { headers: { "Accept": "application/json" } });
  if (!res.ok) throw new Error(`NHL API error: ${res.status}`);
  const data = await res.json();
  return data.data || [];
}

async function buildNHLProjection(playerId: number, statType: string) {
  if (statType !== "Points") return null; // Currently support Points only for NH

  const [mapping] = await db.select().from(playerIdMappingTable)
    .where(and(eq(playerIdMappingTable.playerId, playerId), eq(playerIdMappingTab
    .limit(1);
  if (!mapping) return null;

  const gameLog = await fetchNHLGameLog(mapping.externalId);
  if (!gameLog.length) return null;

  const statValues = gameLog.map((g: any) => (g.goals || 0) + (g.assists || 0));
  const weights = WEIGHTS_10.slice(0, statValues.length);
  const wSum = weights.reduce((a: number, b: number) => a + b, 0);
  const normalized = weights.map((w: number) => w / wSum);
  const weightedAvg = statValues.reduce((sum: number, val: number, i: number) =>
```

```
  return {
    projectedValue: Math.round(weightedAvg * 100) / 100,
    weightedAvg: Math.round(weightedAvg * 100) / 100,
    defenseFactor: 1.0, paceFactor: 1.0,
    gamesUsed: statValues.length,
    confidence: statValues.length >= 8 ? "high" : "medium",
  };
}

// ─── MAIN PROJECTION SYNC ───
export async function syncProjections(): Promise<number> {
  const activeLines = await db
    .select({ line: ppLinesTable, player: playersTable })
    .from(ppLinesTable)
    .innerJoin(playersTable, eq(ppLinesTable.playerId, playersTable.id))
    .where(eq(ppLinesTable.isActive, true));

  let processed = 0;
  for (const { line, player } of activeLines) {
    try {
      let result = null;

      if (player.sport === "NBA" || player.sport === "WNBA") {
        result = await buildNBAProjection(player.id, line.statType);
        await new Promise(r => setTimeout(r, 300)); // rate limit NBA Stats API
      } else if (player.sport === "NHL") {
        result = await buildNHLProjection(player.id, line.statType);
        await new Promise(r => setTimeout(r, 200));
      }

      if (result) {
        await db.insert(ourProjectionsTable).values({
          playerId: player.id,
          statType: line.statType,
          projectedValue: result.projectedValue.toString(),
          weightedAvg: result.weightedAvg.toString(),
          defenseFactor: result.defenseFactor.toString(),
          paceFactor: result.paceFactor.toString(),
          gamesUsed: result.gamesUsed,
          confidence: result.confidence,
          generatedAt: new Date(),
        }).onConflictDoUpdate({
          target: [ourProjectionsTable.playerId, ourProjectionsTable.statType],
          set: {
            projectedValue: result.projectedValue.toString(),
            weightedAvg: result.weightedAvg.toString(),
            gamesUsed: result.gamesUsed,
```

```
            confidence: result.confidence,
            generatedAt: new Date(),
          }
        });
        processed++;
      }
    } catch (e) {
      console.error(`Projection error for ${player.fullName} ${line.statType}:`,
    }
  }
  return processed;
}

// ─── PLAYER ID MAPPING BUILDER ───
// Run once manually, then update incrementally
// Maps player names from PrizePicks to NBA/NHL/MLB numeric IDs
export async function buildPlayerIdMappings(): Promise<void> {
  // NBA players
  const nbaRoster = await fetch(
    "https://stats.nba.com/stats/commonallplayers?IsOnlyCurrentSeason=1&LeagueID=
    { headers: { "User-Agent": "Mozilla/5.0", "Referer": "https://www.nba.com/",
  );
  const nbaData = await nbaRoster.json();
  const headers = nbaData.resultSets[0].headers;
  const rows = nbaData.resultSets[0].rowSet;
  const nbaPlayers = rows.map((row: any[]) => Object.fromEntries(headers.map((h:

  const dbPlayers = await db.select().from(playersTable).where(eq(playersTable.sp
  for (const dbPlayer of dbPlayers) {
    const match = nbaPlayers.find((p: any) => {
      const ppName = dbPlayer.fullName.toLowerCase();
      const nbaName = (p.DISPLAY_FIRST_LAST || "").toLowerCase();
      return ppName === nbaName || ppName.split(" ").pop() === nbaName.split(" ")
    });
    if (match) {
      await db.insert(playerIdMappingTable).values({
        playerId: dbPlayer.id, source: "nba",
        externalId: match.PERSON_ID.toString(),
        externalName: match.DISPLAY_FIRST_LAST,
      }).onConflictDoNothing();
    }
  }
}
```

2D. Result Tracking & CLV

File: artifacts/api-server/src/routes/results.ts

```
import { Router } from "express";
import { db } from "@workspace/db";
import { entryPicksTable, entriesTable, clvRecordsTable, ppLinesTable, probabilit
import { eq, and } from "drizzle-orm";

const router = Router();

// Mark a pick as hit or miss
router.post("/results/pick/:pickId", async (req, res) => {
  try {
    const pickId = Number(req.params.pickId);
    const { result, closingLine } = req.body as { result: "hit" | "miss" | "dnp",

    const [pick] = await db.select().from(entryPicksTable).where(eq(entryPicksTab
    if (!pick) return void res.status(404).json({ error: "Pick not found" });

    // Update pick result
    await db.update(entryPicksTable).set({ result }).where(eq(entryPicksTable.id,

    // Record CLV if closing line provided
    if (closingLine && pick.ppLineId) {
      const [ppLine] = await db.select().from(ppLinesTable).where(eq(ppLinesTable
      if (ppLine) {
        const lockedLine = parseFloat(ppLine.lineValue.toString());
        const direction = pick.direction || "more";
        // CLV: if direction=more, positive CLV means closing line went up (marke
        const clv = direction === "more"
          ? closingLine - lockedLine
          : lockedLine - closingLine;

        await db.insert(clvRecordsTable).values({
          entryPickId: pickId, ppLineId: pick.ppLineId,
          lockedLine: lockedLine.toString(),
          closingLine: closingLine.toString(),
          clv: clv.toString(), direction,
        });
      }
    }

    // Recalibrate probability model for this sport/stat/lineType/edge bucket
    if (result === "hit" || result === "miss") {
      await recalibrateProbability(pick, result);
    }
```

```
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: "Internal server error" });
  }
});

// Get all pending (unresolved) picks — for the "Mark Results" workflow
router.get("/results/pending", async (req, res) => {
  try {
    const pending = await db
      .select({ pick: entryPicksTable, entry: entriesTable, player: playersTable,
      .from(entryPicksTable)
      .innerJoin(entriesTable, eq(entryPicksTable.entryId, entriesTable.id))
      .innerJoin(ppLinesTable, eq(entryPicksTable.ppLineId, ppLinesTable.id))
      .innerJoin(playersTable, eq(ppLinesTable.playerId, playersTable.id))
      .where(eq(entryPicksTable.result, "pending"));
    res.json(pending);
  } catch (err) {
    res.status(500).json({ error: "Internal server error" });
  }
});

// Batch mark results for an entire entry
router.post("/results/entry/:entryId", async (req, res) => {
  try {
    const entryId = Number(req.params.entryId);
    const { picks } = req.body as { picks: Array<{ pickId: number, result: "hit"

    for (const p of picks) {
      await fetch(`/api/results/pick/${p.pickId}`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ result: p.result, closingLine: p.closingLine }),
      });
    }

    // Determine overall entry result
    const results = picks.filter(p => p.result !== "dnp").map(p => p.result);
    const hits = results.filter(r => r === "hit").length;
    const total = results.length;
    const entryResult = hits === total ? "win" : hits === 0 ? "loss" : "partial";

    await db.update(entriesTable).set({ result: entryResult }).where(eq(entriesTa
    res.json({ success: true, entryResult });
  } catch (err) {
    res.status(500).json({ error: "Internal server error" });
  }
```

```
});

async function recalibrateProbability(pick: any, result: "hit" | "miss") {
  try {
    const [ppLine] = await db.select({ line: ppLinesTable, player: playersTable }
      .from(ppLinesTable)
      .innerJoin(playersTable, eq(ppLinesTable.playerId, playersTable.id))
      .where(eq(ppLinesTable.id, pick.ppLineId))
      .limit(1);
    if (!ppLine) return;

    // Get edge score to determine bucket
    const lineType = ppLine.line.lineType;
    const edgeBucket = "mid"; // simplified — in production, look up propScore.ed
    const direction = pick.direction || "more";

    const [existing] = await db.select().from(probabilityCalibrationTable)
      .where(and(
        eq(probabilityCalibrationTable.sport, ppLine.player.sport),
        eq(probabilityCalibrationTable.statType, ppLine.line.statType),
        eq(probabilityCalibrationTable.lineType, lineType),
        eq(probabilityCalibrationTable.edgeBucket, edgeBucket),
        eq(probabilityCalibrationTable.direction, direction),
      )).limit(1);

    const newSample = (existing?.sampleSize || 0) + 1;
    const newHits = (existing?.hitCount || 0) + (result === "hit" ? 1 : 0);
    const newHitRate = newHits / newSample;

    // Wilson confidence interval
    const z = 1.96;
    const ci = z * Math.sqrt((newHitRate * (1 - newHitRate)) / newSample + z*z/(4

    const values = {
      sport: ppLine.player.sport, statType: ppLine.line.statType,
      lineType, edgeBucket, direction,
      sampleSize: newSample, hitCount: newHits,
      hitRate: newHitRate.toString(), confidenceInterval: ci.toString(),
      lastUpdated: new Date(),
    };

    await db.insert(probabilityCalibrationTable).values(values)
      .onConflictDoUpdate({ target: [/* unique constraint fields */], set: values
  } catch (e) { console.error("Calibration error:", e); }
}

export default router;
```

2E. Force Sync + Server-Sent Events (Real-Time)

File: `artifacts/api-server/src/lib/sse.ts`

```
import { Response } from "express";

// SSE client registry
const clients = new Set<Response>();

export function addSSEClient(res: Response) {
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.setHeader("Connection", "keep-alive");
  res.setHeader("Access-Control-Allow-Origin", "*");
  res.flushHeaders();

  // Heartbeat every 25 seconds
  const heartbeat = setInterval(() => {
    res.write("event: heartbeat\ndata: {}\n\n");
  }, 25000);

  clients.add(res);

  res.on("close", () => {
    clearInterval(heartbeat);
    clients.delete(res);
  });
}

export function broadcast(event: string, data: object) {
  const payload = `event: ${event}\ndata: ${JSON.stringify(data)}\n\n`;
  clients.forEach(client => {
    try { client.write(payload); } catch { clients.delete(client); }
  });
}

// Sync status broadcast
export function broadcastSyncStatus(job: string, status: "running" | "success" |
  broadcast("sync_status", { job, status, detail, timestamp: new Date().toISOStri
}

// New goblin line alert
export function broadcastNewGoblin(playerName: string, stat: string, line: number
  broadcast("new_goblin", { playerName, stat, line, sport, timestamp: new Date().
}
```

```
// Line moved alert
export function broadcastLineMove(playerName: string, stat: string, from: number,
  broadcast("line_move", { playerName, stat, from, to, diff: to - from, timestamp
}
```

File: `artifacts/api-server/src/routes/sync.ts` — Replace the existing file entirely:

```
import { Router } from "express";
import { db } from "@workspace/db";
import { dataPullLogsTable, alertsTable } from "@workspace/db/schema";
import { eq } from "drizzle-orm";
import { broadcastSyncStatus } from "../lib/sse";
import { syncPpLines } from "../lib/sync/prizepicks";
import { syncExternalOdds } from "../lib/sync/external-odds";
import { syncProjections } from "../lib/sync/projections";

const router = Router();

async function runSync(provider: string, jobName: string, fn: () => Promise<numbe
  const [log] = await db.insert(dataPullLogsTable).values({
    provider, jobName, status: "running", startedAt: new Date()
  }).returning();

  res.json({ status: "started", logId: log.id });
  broadcastSyncStatus(jobName, "running");

  try {
    const recordsProcessed = await fn();
    await db.update(dataPullLogsTable).set({
      status: "success", recordsProcessed, finishedAt: new Date()
    }).where(eq(dataPullLogsTable.id, log.id));
    broadcastSyncStatus(jobName, "success", `${recordsProcessed} records`);
  } catch (err) {
    const errorMessage = err instanceof Error ? err.message : "Unknown error";
    await db.update(dataPullLogsTable).set({
      status: "error", errorMessage, finishedAt: new Date()
    }).where(eq(dataPullLogsTable.id, log.id));
    broadcastSyncStatus(jobName, "error", errorMessage);
    await db.insert(alertsTable).values({
      type: "sync_failure", severity: "warning",
      title: `Sync Failed: ${jobName}`, message: errorMessage,
    });
  }
}
```

```
router.post("/sync/pp-lines", async (req, res) => runSync("prizepicks", "pp-lines
router.post("/sync/external-odds", async (req, res) => runSync("the-odds-api", "e
router.post("/sync/projections", async (req, res) => runSync("nba-stats", "projec

// FORCE SYNC ALL — triggers all syncs sequentially, used by the UI button
router.post("/sync/all", async (req, res) => {
  res.json({ status: "started", message: "All syncs initiated" });
  broadcastSyncStatus("all", "running");

  const jobs = [
    { name: "pp-lines", fn: syncPpLines },
    { name: "external-odds", fn: syncExternalOdds },
    { name: "projections", fn: syncProjections },
  ];

  for (const job of jobs) {
    broadcastSyncStatus(job.name, "running");
    try {
      const n = await job.fn();
      broadcastSyncStatus(job.name, "success", `${n} records`);
    } catch (e) {
      broadcastSyncStatus(job.name, "error", e instanceof Error ? e.message : "Un
    }
    await new Promise(r => setTimeout(r, 500));
  }

  broadcastSyncStatus("all", "success", "All syncs complete");
});

export default router;
```

Add SSE endpoint to `artifacts/api-server/src/app.ts`:

```
import { addSSEClient } from "./lib/sse";
// Add this route before the main router:
app.get("/api/events", (req, res) => addSSEClient(res));
```

2F. Cron Jobs — Real Implementations

File: `artifacts/api-server/src/lib/cron.ts` — Replace entirely:

```
import cron from "node-cron";
import { db } from "@workspace/db";
import { dataPullLogsTable, alertsTable, userSettingsTable, sessionLogsTable } fr
import { eq, and } from "drizzle-orm";
```

```
import { syncPpLines } from "./sync/prizepicks";
import { syncExternalOdds } from "./sync/external-odds";
import { syncProjections } from "./sync/projections";
import { broadcastSyncStatus, broadcastNewGoblin } from "./sse";
import { logger } from "./logger";

async function logPull(provider: string, jobName: string, fn: () => Promise<numbe
  const [log] = await db.insert(dataPullLogsTable).values({
    provider, jobName, status: "running", startedAt: new Date()
  }).returning();
  broadcastSyncStatus(jobName, "running");
  try {
    const recordsProcessed = await fn();
    await db.update(dataPullLogsTable).set({ status: "success", recordsProcessed,
    broadcastSyncStatus(jobName, "success", `${recordsProcessed} records`);
    logger.info({ provider, jobName, recordsProcessed }, "Sync OK");
  } catch (err) {
    const errorMessage = err instanceof Error ? err.message : "Unknown";
    await db.update(dataPullLogsTable).set({ status: "error", errorMessage, finis
    broadcastSyncStatus(jobName, "error", errorMessage);
    await db.insert(alertsTable).values({ type: "sync_failure", severity: "warnin
    logger.error({ err, provider, jobName }, "Sync failed");
  }
}

export function startCronJobs() {
  cron.schedule("*/10 * * * *", () => logPull("prizepicks", "pp-lines", syncPpLin
  cron.schedule("*/20 * * * *", () => logPull("the-odds-api", "external-odds", sy
  cron.schedule("0 6 * * *", () => logPull("nba-stats", "projections", syncProjec
  cron.schedule("0 7 * * *", () => logPull("nba-stats", "player-id-mappings", asy
    const { buildPlayerIdMappings } = await import("./sync/projections");
    await buildPlayerIdMappings();
    return 1;
  }));

  // Daily loss limit check every 5 minutes
  cron.schedule("*/5 * * * *", async () => {
    try {
      const today = new Date().toISOString().split("T")[0];
      const settings = await db.select().from(userSettingsTable);
      for (const s of settings) {
        if (!s.dailyLossLimit) continue;
        const [session] = await db.select().from(sessionLogsTable)
          .where(and(eq(sessionLogsTable.userId, s.userId), eq(sessionLogsTable.s
        if (session && Number(session.totalPnl) < 0 && Math.abs(Number(session.to
          await db.update(sessionLogsTable).set({ isLocked: true }).where(eq(sess
          await db.insert(alertsTable).values({
```

```
            type: "loss_limit_hit", severity: "critical",
            title: "Daily Loss Limit Reached",
            message: `Daily loss limit of $${s.dailyLossLimit} reached. Entries l
          });
        }
      }
    } catch (e) { logger.error(e, "Loss limit check failed"); }
  });

  logger.info("Cron jobs started");
}
```

2G. Market Intelligence Route

File: `artifacts/api-server/src/routes/market-intel.ts`

```
import { Router } from "express";
import { db } from "@workspace/db";
import { ppLinesTable, externalLinesTable, propScoresTable, playersTable, ourProj
import { eq, and, desc } from "drizzle-orm";

const router = Router();

router.get("/market-intel", async (req, res) => {
  try {
    const rows = await db
      .select({ line: ppLinesTable, player: playersTable, score: propScoresTable,
      .from(ppLinesTable)
      .innerJoin(playersTable, eq(ppLinesTable.playerId, playersTable.id))
      .leftJoin(propScoresTable, eq(propScoresTable.ppLineId, ppLinesTable.id))
      .leftJoin(ourProjectionsTable, and(eq(ourProjectionsTable.playerId, ppLines
      .leftJoin(playerStreaksTable, and(eq(playerStreaksTable.playerId, ppLinesTa
      .where(eq(ppLinesTable.isActive, true));

    const result = await Promise.all(rows.map(async row => {
      const extLines = await db.select().from(externalLinesTable).where(eq(extern
      const bookLines: Record<string, number> = {};
      for (const l of extLines) bookLines[l.bookName] = parseFloat(l.lineValue.to

      const vals = Object.values(bookLines);
      const marketAvg = vals.length ? vals.reduce((a,b)=>a+b,0)/vals.length : nul
      const ppLine = parseFloat(row.line.lineValue.toString());
      const trueEdge = marketAvg ? (-(ppLine - marketAvg) / marketAvg * 100) : nu

      const recentMoves = await db.select().from(lineMoveEventsTable)
        .where(eq(lineMoveEventsTable.ppLineId, row.line.id))
```

```
        .orderBy(desc(lineMoveEventsTable.capturedAt)).limit(5);

      return {
        ppLineId: row.line.id,
        playerId: row.player.id,
        playerName: row.player.fullName,
        team: row.player.team,
        sport: row.player.sport,
        statType: row.line.statType,
        lineValue: ppLine,
        lineType: row.line.lineType,
        marketAvg: marketAvg ? Math.round(marketAvg * 100) / 100 : null,
        trueEdge: trueEdge ? Math.round(trueEdge * 10) / 10 : null,
        bookLines,
        edgeScore: row.score ? parseFloat(row.score.finalScore?.toString() || "0"
        actionTag: row.score?.actionTag || null,
        ourProjection: row.proj ? {
          value: parseFloat(row.proj.projectedValue.toString()),
          confidence: row.proj.confidence,
          gamesUsed: row.proj.gamesUsed,
        } : null,
        streak: row.streak ? {
          count: Math.abs(row.streak.currentStreak || 0),
          type: row.streak.streakType,
        } : null,
        recentMoves: recentMoves.map(m => ({
          book: m.bookName, from: m.prevLine, to: m.newLine,
          direction: m.moveDirection, at: m.capturedAt,
        })),
      };
    }));

    res.json(result.sort((a, b) => (b.trueEdge || 0) - (a.trueEdge || 0)));
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Internal server error" });
  }
});

export default router;
```

2H. User Settings Route

```
// artifacts/api-server/src/routes/user-settings.ts
import { Router } from "express";
import { db } from "@workspace/db";
```

```
import { userSettingsTable } from "@workspace/db/schema";
import { eq } from "drizzle-orm";

const router = Router();
const DEFAULT_SETTINGS = {
  bankroll: "500", unitSize: "25", kellyFraction: "0.25", maxUnitsPerEntry: 1,
  hitRateAssumptions: { goblinOver:0.65, standardHigh:0.58, standardMid:0.52, sta
  payoutConfig: { power: { 2:3, 3:6, 4:10, 5:20, 6:40 } },
  edgeWeights: { marketGap:60, lineType:25, ourProjection:15 },
  excludeDemonOvers: true, minEdgeToPlay: "4",
  aiModel: "claude-sonnet-4-20250514",
};

router.get("/user-settings", async (req, res) => {
  try {
    const userId = (req as any).auth?.userId || "default";
    let [s] = await db.select().from(userSettingsTable).where(eq(userSettingsTabl
    if (!s) {
      [s] = await db.insert(userSettingsTable).values({ userId, ...DEFAULT_SETTIN
    }
    res.json(s);
  } catch { res.status(500).json({ error: "Internal server error" }); }
});

router.put("/user-settings", async (req, res) => {
  try {
    const userId = (req as any).auth?.userId || "default";
    const [s] = await db.update(userSettingsTable)
      .set({ ...req.body, updatedAt: new Date() })
      .where(eq(userSettingsTable.userId, userId))
      .returning();
    res.json(s);
  } catch { res.status(500).json({ error: "Internal server error" }); }
});

export default router;
```

2I. Update Anthropic Route — Full Slate Context + Weapon Modes

In `artifacts/api-server/src/routes/anthropic.ts`, update the system prompt in the message handler:

```
// Add this helper to build slate context before Claude call
async function buildSlateContext(): Promise<string> {
  try {
    const rows = await db
```

```
      .select({ line: ppLinesTable, player: playersTable, score: propScoresTable,
      .from(ppLinesTable)
      .innerJoin(playersTable, eq(ppLinesTable.playerId, playersTable.id))
      .leftJoin(propScoresTable, eq(propScoresTable.ppLineId, ppLinesTable.id))
      .leftJoin(ourProjectionsTable, and(eq(ourProjectionsTable.playerId, ppLines
      .where(eq(ppLinesTable.isActive, true))
      .limit(80);

    return rows.map(r => {
      const proj = r.proj ? ` | OurProj:${r.proj.projectedValue}(${r.proj.confide
      const edge = r.score ? ` | Edge:${r.score.finalScore} [${r.score.actionTag}
      return `${r.player.fullName}(${r.player.team}) — ${r.player.sport} · ${r.li
    }).join("\n");
  } catch { return "Slate context unavailable."; }
}

// Replace the system prompt in the messages handler:
const slateContext = await buildSlateContext();
const systemPrompt = `You are a professional prop trading analyst — not a chatbot

TODAY'S LIVE SLATE:
${slateContext}

RULES:
1. Lead with the single most actionable finding. Never bury the lead.
2. Goblin OVER ≈ 65% hit rate. Demon UNDER ≈ 65%. Demon OVER ≈ 35% — never recomm
3. Positive True Edge = PP line softer than DK/FD market = OVER value.
4. Correlation kills power plays. NBA teammate stacks share possessions. Flag imm
5. GTD players in power plays = entry-destroying risk. Always mention.
6. EV is the only metric that matters long-term. Mention in every lineup evaluati
7. Be honest about uncertainty. If data is thin, say so.
8. Never tell the user what they want to hear. Tell them what the data says.`;
```

2J. Register All New Routes

In `artifacts/api-server/src/routes/index.ts`, add:

```
import marketIntelRouter from "./market-intel";
import userSettingsRouter from "./user-settings";
import resultsRouter from "./results";

router.use(marketIntelRouter);
router.use(userSettingsRouter);
router.use(resultsRouter);
```

PART 3: FRONTEND IMPLEMENTATION

3A. Global: SSE Connection + Force Sync Button

Add to `artifacts/prizepicks/src/App.tsx`:

```
// SSE hook — add near top of file
function useSSE() {
  const qc = useQueryClient();
  const [syncStatus, setSyncStatus] = useState<Record<string, string>>({});
  const [notifications, setNotifications] = useState<any[]>([]);

  useEffect(() => {
    const es = new EventSource("/api/events");
    es.addEventListener("sync_status", (e) => {
      const data = JSON.parse(e.data);
      setSyncStatus(prev => ({ ...prev, [data.job]: data.status }));
      if (data.status === "success") {
        // Invalidate all data queries so UI refreshes automatically
        qc.invalidateQueries();
      }
    });
    es.addEventListener("new_goblin", (e) => {
      const data = JSON.parse(e.data);
      setNotifications(prev => [{ type: "goblin", ...data }, ...prev.slice(0, 9)]
    });
    es.addEventListener("line_move", (e) => {
      const data = JSON.parse(e.data);
      setNotifications(prev => [{ type: "move", ...data }, ...prev.slice(0, 9)]);
    });
    return () => es.close();
  }, [qc]);

  return { syncStatus, notifications };
}
```

3B. Force Sync Button — Add to Sidebar and Dashboard

The sync button appears in TWO places:

1. Bottom of sidebar (always visible)
2. Dashboard header (prominent, with last-sync timestamp)

```
// ForceSync component — add to Layout
function ForceSyncButton({ syncStatus }: { syncStatus: Record<string, string> })
```

```
  const [syncing, setSyncing] = useState(false);
  const { toast } = useToast();

  async function forceSync() {
    setSyncing(true);
    try {
      await fetch("/api/sync/all", { method: "POST" });
      toast({ title: "Sync started", description: "Pulling fresh data from all so
    } catch {
      toast({ title: "Sync failed", variant: "destructive" });
    }
    // SSE will update status and trigger query invalidation
    setTimeout(() => setSyncing(false), 8000);
  }

  const isRunning = syncing || Object.values(syncStatus).some(s => s === "running

  return (
    <button
      onClick={forceSync}
      disabled={isRunning}
      className={`flex items-center gap-2 px-3 py-2 rounded text-xs font-mono tra
        isRunning
          ? "bg-amber-900/30 text-amber-400 border border-amber-700/40 cursor-wai
          : "bg-primary/10 text-primary border border-primary/30 hover:bg-primary
      }`}
    >
      <RefreshCw className={`w-3 h-3 ${isRunning ? "animate-spin" : ""}`} />
      {isRunning ? "Syncing…" : "Force Sync"}
    </button>
  );
}
```

3C. Dashboard — Connect to Real Data

Update `artifacts/prizepicks/src/pages/dashboard.tsx` to use `useGetDashboardSummary` (already wired) and add:

- Force Sync button in header (next to “LIVE DATA” badge)
- SSE notification feed (new goblins, line moves)
- CLV summary widget (7-day and 30-day average)
- Our projection quality indicator (how many props have auto-projections)

3D. Slate Board — Connect to `/api/market-intel`

Replace the `useGetSlate` hook with a direct fetch to `/api/market-intel`. This returns the full enriched slate including:

- Market lines (DK/FD/ESPN)
- True Edge %
- Our own projection (weighted model)
- Streaks
- Line movement history

Add columns to the table:

- **OUR PROJ** — shows value and confidence badge (high/medium/low)
- **PROJ EDGE** — gap between our projection and the line (our model's signal)
- **MKT EDGE** — true edge vs market consensus
- **STREAK** — "6↑ Over" or "3↓ Under" badge
- **MOVES** — mini indicator showing number of recent line movements

Add the **Force Sync** button to the Slate Board header alongside the existing Refresh button.

3E. Entry Builder — Mark Results Workflow

Add a **"Mark Results"** tab to the Entry Builder. This appears when there are pending (unresolved) picks from past entries:

```
PENDING RESULTS                          [Mark All]

Anthony Davis — Points — 26.5 OVER
  [✓ HIT]  [✗ MISS]  [DNP]  Closing line: [___] (optional)

LeBron James — Assists — 7.5 OVER
  [✓ HIT]  [✗ MISS]  [DNP]  Closing line: [___]
```

Submitting results:

1. Marks each pick in the database
2. Records CLV if closing line was provided
3. Triggers probability recalibration
4. Updates entry P&L

5. Updates player streaks

3F. Performance Review — CLV as Primary Metric

Update `artifacts/prizepicks/src/pages/review.tsx`:

Primary KPI row:

1. 7-Day CLV Average (primary metric)
2. 30-Day CLV Average
3. Total P&L (secondary)
4. Pick Hit Rate (by calibration bucket)
5. Optimizer vs. Manual Override comparison

Charts:

1. CLV trend line (30 days) — PRIMARY chart
2. Bankroll curve (secondary)
3. Hit rate by sport/stat type heatmap
4. Manual override performance vs. optimizer

Behavioral insights panel:

- "You override the optimizer X% of the time. Your override hit rate: Y%"
- "Your best stat type: Z (X% hit rate over N picks)"
- "Your worst time to play: Late night (>10pm) entries hit at X%"

3G. Settings Page — Save to Backend

Update `artifacts/prizepicks/src/pages/settings.tsx`:

- On load: `GET /api/user-settings`
- On save: `PUT /api/user-settings`
- Keep existing sync trigger buttons (they already work)
- Add Force Sync All button (prominent, at top of page)
- Add new sections: Hit Rate Assumptions, Payout Config, Edge Weights, Kelly Fraction, Daily Loss Limit

3H. Loss Limit Banner

Add to Layout component. Check SSE for `loss_limit_hit` event OR poll `/api/session-status`. When triggered, show:

```
🔒 Daily Loss Limit Reached — Sessions locked. Trading resumes tomorrow. [I Under
```

Override requires typing "CONFIRM" to proceed. This creates friction that protects the user from tilt.

3I. New Pages to Add

Streak Tracker (`/streaks`)

- Players on 4+ consecutive OVER or UNDER streaks
- Sorted by streak length
- Shows today's line vs. streak context
- Quick-add to entry

Matchup Analysis (`/matchup`)

- Head-to-head performance: player vs. specific opponent
- Pulls from `matchup_history` table (populated from historical logged picks)
- Shows avg value, over rate at current line, last 5 meetings

CLV Tracker (`/clv`)

- All time CLV records, grouped by sport/stat/strategy
- CLV trend chart
- Win/loss record broken out by CLV bucket (positive CLV picks vs. negative CLV picks)

User Guide (`/guide`)

- Embed the full VibeMeGood User Guide as a searchable, scrollable page
- Left sidebar table of contents
- Show as default landing page for first-time users (check `localStorage.getItem('hasSeenGuide')`)

PART 4: AI ANALYST — WEAPON MODES

Update the Anthropic route to support different analytical modes via a `mode` query parameter:

```
router.post("/anthropic/conversations/:id/messages", async (req, res) => {
  const { content, mode } = req.body; // mode: analyst | anomaly | contradiction

  const slateContext = await buildSlateContext();

  const systemPrompts = {
    analyst: `You are a professional prop trading analyst. Lead with actionable f
LIVE SLATE: ${slateContext}
RULES: Goblin OVER ≈ 65% hit. Demon UNDER ≈ 65%. Demon OVER ≈ 35% — never recomme

    anomaly: `Scan today's slate for market anomalies. Identify: unusually large
SLATE: ${slateContext}`,

    contradiction: `You are a risk analyst. Find contradictions and hidden danger
Scan for: PLAY picks with GTD/questionable players, teammate stacks in top plays,
Format: [RISK TYPE] — Player — Explanation — Recommendation.
SLATE: ${slateContext}`,

    stresstest: `Stress-test the user's entry before they lock it in.
1. Find the single most likely pick to fail and exactly why.
2. Identify hidden correlations between picks.
3. State the true win probability accounting for correlations.
4. Recommend: submit as-is, swap one pick, or rebuild.
If rebuild: state exactly which pick to replace and with what.
Be direct. Do not soften anything. Money is on the line.
SLATE: ${slateContext}`,
  };

  const systemPrompt = systemPrompts[mode as keyof typeof systemPrompts] || syste
  // ... rest of existing Anthropic call logic
});
```

PART 5: COMPLETE FEATURE CHECKLIST

Infrastructure

- PP line sync — real implementation

- Odds API sync — real implementation
- Our own projection model (NBA Stats API + NHL API)
- Player ID mapping table + builder function
- All cron jobs with real logic
- Server-Sent Events (SSE) for real-time updates
- Force Sync endpoint (`POST /api/sync/all`)
- Force Sync button — sidebar + Dashboard + Slate Board

Analytics Engine

- True Edge calculation (PP vs. DK/FD/ESPN market average)
- Our projection signal (OurProj vs. line gap)
- Combined edge score (market + line type + our projection, weighted)
- Hit probability per pick (configurable, calibration-aware)
- EV calculator (power and flex, accounting for correlation)
- Kelly criterion stake sizing
- Probability calibration table (self-updates after each result)
- Confidence intervals on all probability estimates

Optimizer

- 4 strategies: Goblin Hunter, Demon Under, Balanced, Max EV
- Combination search (top 25 props, up to C(25,6))
- Correlation-adjusted win probability
- Stack penalty for teammates
- Kelly stake recommendation per combination
- “Load into Entry Builder” action

Entry Builder

- OVER/UNDER direction toggle per pick
- Real-time EV display

- Probability chain visualization
- Stack correlation warning
- Entry quality score (0–100)
- Kelly recommendation
- Mark Results workflow (hit/miss/DNP + closing line)

AI Analyst

- 4 analytical modes (Analyst, Anomaly Hunt, Contradiction Check, Stress Test)
- Full slate context injected (including our projections and market lines)
- Conversation history persisted to DB
- Entry context auto-included when user has active picks
- Configurable model (Sonnet vs Haiku) from Settings

Data & Learning

- Result tracking (hit/miss/DNP on every pick)
- CLV recording (locked line vs. closing line)
- Probability self-calibration after each result
- Player streak tracking (update after results)
- Behavioral logging (time, stake vs. Kelly, deviation from optimizer)
- Line movement event logging

Pages

- Dashboard — real data, Force Sync button, CLV widget, SSE notifications
- Slate Board — market-intel endpoint, all columns, Force Sync button
- Optimizer — connected to backend (or run client-side with market-intel data)
- Entry Builder — real EV, Mark Results workflow
- AI Analyst — 4 weapon modes
- Injuries & News — real injury data
- Performance Review — CLV as primary metric

- Settings — persisted to DB, sync triggers
- Streak Tracker — new page
- CLV Tracker — new page
- User Guide — embedded, searchable
- Loss Limit Banner — active when session locked

PART 6: IMPLEMENTATION ORDER

Build in this exact sequence. Each phase depends on the previous.

```
WEEK 1 — DATA FOUNDATION
  1. Run DB migrations (add all new tables)
  2. Implement syncPpLines() — get live props into the database
  3. Implement syncExternalOdds() — get market lines in
  4. Implement recalcPropScores() — calculate edge scores
  5. Connect Slate Board to /api/market-intel
  6. Verify: real props with real edge scores showing in UI

WEEK 2 — PROJECTIONS
  1. Build buildPlayerIdMappings() — run once to link PP names to NBA IDs
  2. Implement NBA projection model — weighted L10 + defense factor
  3. Implement NHL projection model
  4. Connect our projections to market-intel route
  5. Add OUR PROJ column to Slate Board

WEEK 3 — REAL-TIME & SYNC
  1. Implement SSE server
  2. Add /api/sync/all Force Sync endpoint
  3. Add Force Sync buttons (sidebar + Dashboard + Slate Board)
  4. Verify: clicking Force Sync triggers real data refresh in <30 seconds

WEEK 4 — ENTRY & RESULTS
  1. Implement results routes (mark hit/miss/DNP)
  2. Build Mark Results UI in Entry Builder
  3. Wire CLV recording to result marking
  4. Wire probability recalibration to result marking
  5. Verify: marking picks triggers calibration updates

WEEK 5 — AI WEAPON
  1. Update Anthropic route with 4 system prompt modes
  2. Build slate context builder from real DB data
```

```
  3. Update AI Analyst page with mode selector buttons
  4. Test all 4 modes with live slate data

WEEK 6 — OPTIMIZER & ADVANCED PAGES
  1. Connect Optimizer to real prop data
  2. Add Streak Tracker page
  3. Add CLV Tracker page
  4. Update Performance Review with CLV primary metric
  5. Add Loss Limit banner and session locking

WEEK 7 — POLISH & GUIDE
  1. Embed User Guide as searchable page
  2. Behavioral logging on entry submission
  3. Settings page fully connected to backend
  4. End-to-end test of complete workflow
```

PART 7: TESTING CHECKLIST

Before marking complete:

```
DATA
[ ] /api/market-intel returns real props (not empty array)
[ ] External lines have 2+ bookmakers per prop (after ODDS_API_KEY is set)
[ ] Our projections show for NBA props with confidence ratings
[ ] Edge scores are calculated (not all null)

SYNC
[ ] Force Sync button triggers visible status update in <5 seconds
[ ] SSE events received in browser (check DevTools → Network → EventStream)
[ ] After Force Sync, Slate Board data updates without page refresh

RESULTS WORKFLOW
[ ] Marking a pick as HIT/MISS saves to database
[ ] CLV is recorded when closing line is provided
[ ] Entry result auto-calculates (win/loss/partial) after all picks marked
[ ] Probability calibration table updates after pick is marked

AI ANALYST
[ ] Slate context (real player names and lines) is included in every Claude call
[ ] All 4 modes return distinct, useful responses
[ ] Response quality improves when picks are in entry context

SETTINGS
[ ] Settings save to database (not just local state)
```

```
[ ] Bankroll and unit size reflect in Entry Builder Kelly calculation
[ ] Loss limit locks session when triggered

OPTIMIZER
[ ] All 4 strategies return 10 combinations
[ ] Goblin Hunter returns only goblin OVER picks
[ ] Demon Under returns only demon picks in UNDER direction
[ ] "Load into Entry Builder" populates picks correctly
```

PART 8: COST BREAKDOWN (Monthly)

Service	Tier	Cost
Replit	Pro (always-on)	$25/mo
PostgreSQL	Neon free tier	$0/mo
The Odds API	Free (500 req/mo)	$0/mo
NBA Stats API	Free (no key)	$0/mo
NHL API	Free (no key)	$0/mo
Anthropic Claude Sonnet	~50 sessions/mo	$5–10/mo
Clerk Auth	Free (1 user)	$0/mo
Total		**~$30–35/mo**

If Anthropic costs exceed $10/mo, switch AI model to Haiku in Settings ($1/mo equivalent).

PHILOSOPHY — READ THIS LAST

This platform exists to answer one question:

Does the user make better decisions over time?

Not: do they win more short-term. Not: do they use the app more. Better decisions. Measured by CLV trending positive over 90+ days.

Every feature either finds an edge, quantifies it precisely, or protects the user from destroying it. If a feature doesn't do one of those three things, it shouldn't be in this app.

The user is competing against recreational players who make emotional decisions. This platform makes them systematic. That is the edge.

PART 9: COMPLETE PROJECTION IMPLEMENTATIONS — ALL SPORTS

> These replace the stub `syncProjections()` in the spec above. Add all of this to `artifacts/api-server/src/lib/sync/projections.ts`.

9A. MLB FULL PROJECTION MODEL

Uses the official MLB Stats API (free, no key required). Includes:

- Recency-weighted L10 game log
- Probable pitcher adjustment (ERA + WHIP)
- Ballpark run factor (every stadium, updated for 2025-26)

Ballpark Factors

```
// Park factors relative to league average (1.0 = neutral)
// Higher = more offense, Lower = more pitcher-friendly
// Update these at the start of each season
const PARK_FACTORS: Record<string, number> = {
  "COL": 1.15,  // Coors Field — extreme hitter's park (altitude)
  "CIN": 1.09,  // Great American Ball Park
  "BOS": 1.08,  // Fenway Park — Green Monster inflates doubles
  "PHI": 1.06,  // Citizens Bank Park
  "BAL": 1.05,  // Camden Yards
  "NYY": 1.04,  // Yankee Stadium — short porch in right
  "CHC": 1.03,  // Wrigley Field — wind-dependent, use 1.03 as base
  "HOU": 1.02,  // Minute Maid Park
  "TOR": 1.02,  // Rogers Centre
  "ATL": 1.01,  // Truist Park
  "ARI": 1.00,  // Chase Field
  "MIL": 0.99,  // American Family Field
  "STL": 0.98,  // Busch Stadium
  "WSH": 0.98,  // Nationals Park
```

```
  "LAA": 0.97,  // Angel Stadium
  "DET": 0.97,  // Comerica Park
  "MIN": 0.96,  // Target Field
  "CLE": 0.96,  // Progressive Field
  "KCR": 0.95,  // Kauffman Stadium
  "LAD": 0.94,  // Dodger Stadium — pitcher-friendly
  "CHW": 0.94,  // Guaranteed Rate Field
  "PIT": 0.93,  // PNC Park
  "NYM": 0.93,  // Citi Field
  "OAK": 0.93,  // Oakland Coliseum
  "TEX": 0.92,  // Globe Life Field — retractable roof, climate controlled
  "SFG": 0.92,  // Oracle Park — marine layer suppresses offense
  "SEA": 0.91,  // T-Mobile Park
  "TBR": 0.91,  // Tropicana Field
  "MIA": 0.90,  // loanDepot Park — retractable roof
  "SDP": 0.88,  // Petco Park — most pitcher-friendly park in NL
};

// Ballpark GPS coordinates for weather lookup
const BALLPARK_COORDS: Record<string, { lat: number; lon: number; name: string }>
  "COL": { lat: 39.7559, lon: -104.9942, name: "Coors Field" },
  "BOS": { lat: 42.3467, lon: -71.0972, name: "Fenway Park" },
  "CHC": { lat: 41.9484, lon: -87.6553, name: "Wrigley Field" },
  "NYY": { lat: 40.8296, lon: -73.9262, name: "Yankee Stadium" },
  "NYM": { lat: 40.7571, lon: -73.8458, name: "Citi Field" },
  "LAD": { lat: 34.0739, lon: -118.2400, name: "Dodger Stadium" },
  "SFG": { lat: 37.7786, lon: -122.3893, name: "Oracle Park" },
  "CHW": { lat: 41.8300, lon: -87.6338, name: "Guaranteed Rate Field" },
  "CIN": { lat: 39.0979, lon: -84.5082, name: "GABP" },
  "CLE": { lat: 41.4962, lon: -81.6852, name: "Progressive Field" },
  "DET": { lat: 42.3390, lon: -83.0485, name: "Comerica Park" },
  "HOU": { lat: 29.7573, lon: -95.3555, name: "Minute Maid Park" },
  "KCR": { lat: 39.0517, lon: -94.4803, name: "Kauffman Stadium" },
  "LAA": { lat: 33.8003, lon: -117.8827, name: "Angel Stadium" },
  "MIN": { lat: 44.9817, lon: -93.2778, name: "Target Field" },
  "OAK": { lat: 37.7516, lon: -122.2005, name: "Oakland Coliseum" },
  "SEA": { lat: 47.5914, lon: -122.3325, name: "T-Mobile Park" },
  "TEX": { lat: 32.7512, lon: -97.0832, name: "Globe Life Field" },
  "TOR": { lat: 43.6414, lon: -79.3894, name: "Rogers Centre" },
  "ATL": { lat: 33.8908, lon: -84.4678, name: "Truist Park" },
  "BAL": { lat: 39.2838, lon: -76.6215, name: "Camden Yards" },
  "MIA": { lat: 25.7781, lon: -80.2197, name: "loanDepot Park" },
  "MIL": { lat: 43.0280, lon: -87.9712, name: "American Family Field" },
  "PHI": { lat: 39.9061, lon: -75.1665, name: "Citizens Bank Park" },
  "PIT": { lat: 40.4469, lon: -80.0057, name: "PNC Park" },
  "SDP": { lat: 32.7076, lon: -117.1570, name: "Petco Park" },
  "STL": { lat: 38.6226, lon: -90.1928, name: "Busch Stadium" },
```

```
  "WSH": { lat: 38.8730, lon: -77.0074, name: "Nationals Park" },
  "ARI": { lat: 33.4453, lon: -112.0667, name: "Chase Field" },
  "TBR": { lat: 27.7683, lon: -82.6534, name: "Tropicana Field" },
};
```

MLB Fetch Functions

```
const MLB_BASE = process.env.MLB_STATS_BASE || "https://statsapi.mlb.com/api/v1";
const MLB_HEADERS = { "User-Agent": "Mozilla/5.0", "Accept": "application/json" }

async function fetchMLBGameLog(mlbId: string, numGames = 10): Promise<any[]> {
  const season = "2026";
  const url = `${MLB_BASE}/people/${mlbId}/stats?stats=gameLog&season=${season}&g
  const res = await fetch(url, { headers: MLB_HEADERS });
  if (!res.ok) throw new Error(`MLB game log ${res.status}`);
  const data = await res.json();
  return (data.stats?.[0]?.splits || []).slice(0, numGames);
}

async function fetchMLBPitcherGameLog(mlbId: string, numGames = 5): Promise<any[]
  const url = `${MLB_BASE}/people/${mlbId}/stats?stats=gameLog&season=2026&group=
  const res = await fetch(url, { headers: MLB_HEADERS });
  if (!res.ok) return [];
  const data = await res.json();
  return (data.stats?.[0]?.splits || []).slice(0, numGames);
}

async function fetchTodayMLBGames(): Promise<any[]> {
  const today = new Date().toISOString().split("T")[0];
  const url = `${MLB_BASE}/schedule?sportId=1&date=${today}&hydrate=probablePitch
  const res = await fetch(url, { headers: MLB_HEADERS });
  if (!res.ok) return [];
  const data = await res.json();
  return data.dates?.[0]?.games || [];
}

async function fetchPitcherSeasonStats(mlbId: string): Promise<Record<string, any
  const url = `${MLB_BASE}/people/${mlbId}/stats?stats=season&season=2026&group=p
  const res = await fetch(url, { headers: MLB_HEADERS });
  if (!res.ok) return {};
  const data = await res.json();
  return data.stats?.[0]?.splits?.[0]?.stat || {};
}

async function fetchBatterInfo(mlbId: string): Promise<{ batSide: string; fullNam
  const url = `${MLB_BASE}/people/${mlbId}`;
```

```
  const res = await fetch(url, { headers: MLB_HEADERS });
  if (!res.ok) return { batSide: "R", fullName: "" };
  const data = await res.json();
  const person = data.people?.[0] || {};
  return { batSide: person.batSide?.code || "R", fullName: person.fullName || ""
}

// Build a map of team → probable pitcher for today
async function buildTodayPitcherMap(): Promise<Record<string, { id: string; name:
  const games = await fetchTodayMLBGames();
  const map: Record<string, any> = {};
  for (const game of games) {
    const homeAbbr = game.teams?.home?.team?.abbreviation || "";
    const awayAbbr = game.teams?.away?.team?.abbreviation || "";
    const venue = game.venue?.name || "";
    const homePitcher = game.teams?.home?.probablePitcher;
    const awayPitcher = game.teams?.away?.probablePitcher;
    // Away pitcher faces home batters; home pitcher faces away batters
    if (awayPitcher) {
      map[homeAbbr] = { id: awayPitcher.id?.toString(), name: awayPitcher.fullNam
    }
    if (homePitcher) {
      map[awayAbbr] = { id: homePitcher.id?.toString(), name: homePitcher.fullNam
    }
  }
  return map;
}
```

MLB Projection Builder

```
// Map PP stat types to MLB Stats API keys
const MLB_STAT_MAP: Record<string, string> = {
  "Hits":        "hits",
  "Total Bases": "totalBases",
  "RBIs":        "rbi",
  "Runs":        "runs",
  "Home Runs":   "homeRuns",
  "Doubles":     "doubles",
  "Strikeouts":  "strikeOuts",  // batter strikeouts
};

// League averages 2025-26 (update each season)
const MLB_LEAGUE_AVG = { ERA: 4.20, WHIP: 1.28 };

export async function buildMLBProjection(
  playerId: number,
```

```
  statType: string,
  playerTeam: string,
): Promise<ReturnType<typeof buildNBAProjection> | null> {
  const [mapping] = await db.select().from(playerIdMappingTable)
    .where(and(eq(playerIdMappingTable.playerId, playerId), eq(playerIdMappingTab
    .limit(1);
  if (!mapping) return null;

  const statKey = MLB_STAT_MAP[statType];
  if (!statKey) return null;

  // 1. Batter L10 game log
  const gameLog = await fetchMLBGameLog(mapping.externalId, 10);
  if (!gameLog.length) return null;

  const statValues = gameLog.map((g: any) => parseInt(g.stat?.[statKey] || "0"));
  const weights = WEIGHTS_10.slice(0, statValues.length);
  const wSum = weights.reduce((a: number, b: number) => a + b, 0);
  const normalized = weights.map((w: number) => w / wSum);
  const weightedAvg = statValues.reduce((sum: number, val: number, i: number) =>

  // 2. Pitcher adjustment
  let pitcherFactor = 1.0;
  let pitcherName = "TBD";
  let homeTeam = playerTeam; // default assumption

  try {
    const pitcherMap = await buildTodayPitcherMap();
    const pitcher = pitcherMap[playerTeam];
    if (pitcher?.id) {
      pitcherName = pitcher.name;
      homeTeam = pitcher.homeTeam;
      const stats = await fetchPitcherSeasonStats(pitcher.id);
      const era = parseFloat(stats.era?.toString() || "4.20");
      const whip = parseFloat(stats.whip?.toString() || "1.28");
      const eraFactor  = era  / MLB_LEAGUE_AVG.ERA;   // >1 = weak pitcher = more
      const whipFactor = whip / MLB_LEAGUE_AVG.WHIP;
      pitcherFactor = (eraFactor * 0.6 + whipFactor * 0.4); // weight ERA more he
      pitcherFactor = Math.max(0.65, Math.min(1.45, pitcherFactor)); // cap extre
    }
  } catch { /* non-fatal — use neutral factor */ }

  // 3. Ballpark factor
  const rawParkFactor = PARK_FACTORS[homeTeam] || 1.0;
  // Total bases and hits benefit more directly from park; apply sqrt dampening t
  const parkFactor = ["Total Bases", "Hits", "Home Runs"].includes(statType)
    ? rawParkFactor
```

```
    : 0.5 + rawParkFactor * 0.5; // dampened for counting stats less park-depende

  // 4. Final projection
  const projection = weightedAvg * pitcherFactor * parkFactor;
  const confidence = gameLog.length >= 8 && pitcherName !== "TBD" ? "medium" : "l

  return {
    projectedValue: Math.round(projection * 100) / 100,
    weightedAvg:    Math.round(weightedAvg * 100) / 100,
    defenseFactor:  Math.round(pitcherFactor * 1000) / 1000,
    paceFactor:     Math.round(parkFactor * 1000) / 1000,
    gamesUsed:      gameLog.length,
    confidence,
    notes:          pitcherName !== "TBD" ? `vs. ${pitcherName} · ${homeTeam} par
  };
}
```

MLB Player ID Mapping

```
export async function buildMLBPlayerIdMappings(): Promise<void> {
  const url = `${MLB_BASE}/sports/1/players?season=2026&gameType=R`;
  const res = await fetch(url, { headers: MLB_HEADERS });
  if (!res.ok) throw new Error("MLB roster fetch failed");
  const data = await res.json();
  const mlbPlayers: any[] = data.people || [];

  const dbPlayers = await db.select().from(playersTable).where(eq(playersTable.sp
  let mapped = 0;
  for (const dbPlayer of dbPlayers) {
    const ppName = dbPlayer.fullName.toLowerCase().trim();
    // 1. Exact match
    let match = mlbPlayers.find(p => (p.fullName || "").toLowerCase().trim() ===
    // 2. Last name + first initial match (handles "Mike" vs "Michael")
    if (!match) {
      const ppParts = ppName.split(" ");
      const ppFirst = ppParts[0]?.[0] || "";
      const ppLast  = ppParts.slice(-1)[0] || "";
      match = mlbPlayers.find(p => {
        const n = (p.fullName || "").toLowerCase().split(" ");
        return n.slice(-1)[0] === ppLast && (n[0]?.[0] || "") === ppFirst && ppLa
      });
    }
    if (match) {
      await db.insert(playerIdMappingTable).values({
        playerId: dbPlayer.id, source: "mlb",
        externalId: match.id.toString(), externalName: match.fullName,
```

```
      }).onConflictDoNothing();
      mapped++;
    }
  }
  console.log(`MLB mapping: ${mapped}/${dbPlayers.length} players matched`);
}
```

9B. NFL PROJECTION MODEL

Uses the ESPN unofficial API (free, no key needed). NFL season runs September–February. Build this now so it's ready when the season starts.

> **Note:** The ESPN API is unofficial and subject to change. It has been stable for years but may require header updates if ESPN changes their endpoints.

```
const ESPN_NFL_BASE = "https://site.api.espn.com/apis/site/v2/sports/football/nfl
const ESPN_HEADERS = {
  "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.
  "Accept": "application/json",
};

// Map PP stat types to ESPN stat categories
const NFL_STAT_MAP: Record<string, { category: string; statName: string }> = {
  "Passing Yards":    { category: "passing",   statName: "passingYards" },
  "Rushing Yards":    { category: "rushing",   statName: "rushingYards" },
  "Receiving Yards":  { category: "receiving", statName: "receivingYards" },
  "Receptions":       { category: "receiving", statName: "receptions" },
  "Passing TDs":      { category: "passing",   statName: "passingTouchdowns" },
  "Rushing Attempts": { category: "rushing",   statName: "rushingAttempts" },
  "Targets":          { category: "receiving", statName: "receivingTargets" },
};

async function fetchNFLPlayerGameLog(espnPlayerId: string, numGames = 8): Promise
  const url = `${ESPN_NFL_BASE}/athletes/${espnPlayerId}/gamelog`;
  const res = await fetch(url, { headers: ESPN_HEADERS });
  if (!res.ok) throw new Error(`ESPN NFL gamelog ${res.status}`);
  const data = await res.json();
  // ESPN returns a table structure — parse it
  const categories = data.categories || [];
  const events = data.events || {};
  const results: Record<string, number>[] = [];

  // Build game-by-game stat objects
  const eventKeys = Object.keys(events);
```

```
  for (const key of eventKeys.slice(0, numGames)) {
    const event = events[key];
    const statObj: Record<string, number> = {};
    for (const cat of categories) {
      const catName = cat.name;
      const stats = cat.events?.[key] || [];
      cat.names?.forEach((name: string, i: number) => {
        statObj[name] = parseFloat(stats[i] || "0");
      });
    }
    results.push(statObj);
  }
  return results;
}

async function fetchNFLPlayerESPNId(playerName: string): Promise<string | null> {
  // Search ESPN for player
  const query = encodeURIComponent(playerName);
  const url = `${ESPN_NFL_BASE}/athletes?limit=5&search=${query}`;
  const res = await fetch(url, { headers: ESPN_HEADERS });
  if (!res.ok) return null;
  const data = await res.json();
  const athletes = data.items || [];
  if (!athletes.length) return null;
  // Return first match
  const id = athletes[0]?.id;
  return id?.toString() || null;
}

export async function buildNFLProjection(playerId: number, statType: string): Pro
  const [mapping] = await db.select().from(playerIdMappingTable)
    .where(and(eq(playerIdMappingTable.playerId, playerId), eq(playerIdMappingTab
    .limit(1);
  if (!mapping) return null;

  const statInfo = NFL_STAT_MAP[statType];
  if (!statInfo) return null;

  const gameLog = await fetchNFLPlayerGameLog(mapping.externalId, 8);
  if (!gameLog.length) return null;

  const statValues = gameLog
    .map((g: any) => parseFloat(g[statInfo.statName] || "0"))
    .filter((v: number) => !isNaN(v));
  if (!statValues.length) return null;

  const weights = WEIGHTS_10.slice(0, statValues.length);
```

```
  const wSum = weights.reduce((a, b) => a + b, 0);
  const normalized = weights.map(w => w / wSum);
  const weightedAvg = statValues.reduce((sum, val, i) => sum + val * (normalized[

  return {
    projectedValue: Math.round(weightedAvg * 10) / 10,
    weightedAvg:    Math.round(weightedAvg * 10) / 10,
    defenseFactor:  1.0, // Opponent defense adjustment — future enhancement
    paceFactor:     1.0,
    gamesUsed:      statValues.length,
    confidence:     statValues.length >= 6 ? "medium" : "low",
    notes:          "NFL — L8 weighted avg. Opponent defense adjustment coming.",
  };
}

export async function buildNFLPlayerIdMappings(): Promise<void> {
  const dbPlayers = await db.select().from(playersTable).where(eq(playersTable.sp
  for (const player of dbPlayers) {
    try {
      const espnId = await fetchNFLPlayerESPNId(player.fullName);
      if (espnId) {
        await db.insert(playerIdMappingTable).values({
          playerId: player.id, source: "nfl",
          externalId: espnId, externalName: player.fullName,
        }).onConflictDoNothing();
      }
      await new Promise(r => setTimeout(r, 400)); // rate limit ESPN
    } catch { /* continue */ }
  }
}
```

9C. WEATHER INTEGRATION FOR OUTDOOR GAMES

Weather matters significantly for outdoor MLB and NFL games. Wind affects total bases, hits, and strikeouts in MLB. Wind and temperature affect passing yards in NFL.

Add to environment variables: `OPENWEATHER_API_KEY` (free tier: 1,000 calls/day — plenty)

```
// artifacts/api-server/src/lib/sync/weather.ts

const OW_BASE = "https://api.openweathermap.org/data/2.5";
const OW_KEY = process.env.OPENWEATHER_API_KEY || "";

// Stadiums with roofs — skip weather for these
```

```
const ROOFED_STADIUMS = new Set([
  "TOR", "TBR", "MIA", "HOU", "ARI", "TEX", "MIL", "SEA", // MLB retractable/dome
  // NFL indoor stadiums added when season starts
]);

interface WeatherConditions {
  tempF: number;
  windSpeedMph: number;
  windDirection: number; // degrees: 0=N, 90=E, 180=S, 270=W
  windDirectionLabel: string;
  isIndoor: boolean;
  weatherImpact: "neutral" | "pitcher_friendly" | "hitter_friendly" | "passing_su
  impactScore: number; // -2 to +2, negative = suppresses offense
  notes: string;
}

async function fetchWeather(lat: number, lon: number): Promise<WeatherConditions
  if (!OW_KEY) return null;
  const url = `${OW_BASE}/weather?lat=${lat}&lon=${lon}&appid=${OW_KEY}&units=imp
  const res = await fetch(url);
  if (!res.ok) return null;
  const data = await res.json();

  const tempF = data.main?.temp || 70;
  const windSpeedMph = (data.wind?.speed || 0) * 0.868976; // m/s to mph... actua
  const windDirection = data.wind?.deg || 0;

  const dirLabels = ["N","NE","E","SE","S","SW","W","NW"];
  const dirIdx = Math.round(windDirection / 45) % 8;
  const windDirectionLabel = dirLabels[dirIdx];

  // Calculate impact
  let impactScore = 0;
  const notes: string[] = [];

  // Temperature impact (cold suppresses offense)
  if (tempF < 45) { impactScore -= 1.5; notes.push(`Cold (${tempF.toFixed(0)}°F)
  else if (tempF < 55) { impactScore -= 0.8; notes.push(`Cool (${tempF.toFixed(0)
  else if (tempF > 85) { impactScore += 0.5; notes.push(`Hot (${tempF.toFixed(0)}

  // Wind impact
  if (windSpeedMph >= 15) {
    // Wrigley Field: wind out to center = home run paradise
    // Wind in = pitcher's park. Direction matters enormously at Wrigley specific
    if (windSpeedMph >= 20) {
      impactScore += 1.0; // High wind increases variance regardless of direction
      notes.push(`Strong wind ${windSpeedMph.toFixed(0)}mph ${windDirectionLabel}
```

```
    } else {
      notes.push(`Wind ${windSpeedMph.toFixed(0)}mph ${windDirectionLabel}`);
    }
  }

  const weatherImpact =
    impactScore > 1   ? "hitter_friendly" :
    impactScore < -1  ? "pitcher_friendly" :
    impactScore < -0.5 ? "pitcher_friendly" : "neutral";

  return { tempF, windSpeedMph, windDirection, windDirectionLabel, isIndoor: fals
}

export async function enrichGameWithWeather(): Promise<number> {
  const today = new Date().toISOString().split("T")[0];
  const games = await db.select().from(gamesTable);
  let updated = 0;

  for (const game of games) {
    const homeTeam = await db.select().from(teamsTable).where(eq(teamsTable.id, g
    if (!homeTeam.length) continue;

    const abbr = homeTeam[0].abbreviation || "";
    const isIndoor = ROOFED_STADIUMS.has(abbr);

    let weather: WeatherConditions | null = null;
    if (!isIndoor) {
      const coords = BALLPARK_COORDS[abbr];
      if (coords) {
        weather = await fetchWeather(coords.lat, coords.lon);
        await new Promise(r => setTimeout(r, 200)); // rate limit
      }
    }

    await db.insert(gameEnvironmentTable).values({
      gameId: game.id,
      environmentScore: weather ? Math.round(50 + weather.impactScore * 15) : 50,
      weatherConditions: weather
        ? { tempF: weather.tempF, windSpeedMph: weather.windSpeedMph, windDirecti
        : { isIndoor, notes: isIndoor ? "Retractable roof — weather neutral" : "W
    }).onConflictDoUpdate({
      target: [gameEnvironmentTable.gameId],
      set: {
        environmentScore: weather ? Math.round(50 + weather.impactScore * 15) : 5
        weatherConditions: weather || {},
        updatedAt: new Date(),
      }
```

```
      });
      updated++;
    }
    return updated;
  }
```

Add weather sync to the Force Sync and cron:

In `artifacts/api-server/src/routes/sync.ts`, add:

```
import { enrichGameWithWeather } from "../lib/sync/weather";
// Add to SYNC_JOBS in Force Sync:
{ name: "weather", fn: enrichGameWithWeather },
```

In `artifacts/api-server/src/lib/cron.ts`, add:

```
import { enrichGameWithWeather } from "./sync/weather";
// Runs every hour — weather changes throughout the day
cron.schedule("0 * * * *", () => logPull("openweather", "weather", enrichGameWith
```

Add to environment variables:

```
OPENWEATHER_API_KEY=    # openweathermap.org — free tier, no credit card required
```

9D. UPDATED COMPLETE syncProjections FUNCTION

Replace the partial implementation above with this complete version handling all 4 sports:

```
export async function syncProjections(): Promise<number> {
  const activeLines = await db
    .select({ line: ppLinesTable, player: playersTable })
    .from(ppLinesTable)
    .innerJoin(playersTable, eq(ppLinesTable.playerId, playersTable.id))
    .where(eq(ppLinesTable.isActive, true));

  let processed = 0;
  const errors: string[] = [];

  // Cache today's pitcher map — fetch once, use for all batters
  let mlbPitcherMap: Awaited<ReturnType<typeof buildTodayPitcherMap>> | null = nu

  for (const { line, player } of activeLines) {
```

```
try {
  let result = null;

  if (player.sport === "NBA" || player.sport === "WNBA") {
    result = await buildNBAProjection(player.id, line.statType);
    await new Promise(r => setTimeout(r, 350));
  } else if (player.sport === "NHL") {
    result = await buildNHLProjection(player.id, line.statType);
    await new Promise(r => setTimeout(r, 250));
  } else if (player.sport === "MLB") {
    // Lazy-load pitcher map once per sync
    if (!mlbPitcherMap) {
      mlbPitcherMap = await buildTodayPitcherMap();
    }
    result = await buildMLBProjection(player.id, line.statType, player.team |
    await new Promise(r => setTimeout(r, 300));
  } else if (player.sport === "NFL") {
    result = await buildNFLProjection(player.id, line.statType);
    await new Promise(r => setTimeout(r, 400));
  }

  if (result) {
    await db.insert(ourProjectionsTable).values({
      playerId: player.id,
      statType: line.statType,
      projectedValue: result.projectedValue.toString(),
      weightedAvg:    (result.weightedAvg || result.projectedValue).toString(
      defenseFactor:  (result.defenseFactor || 1).toString(),
      paceFactor:     (result.paceFactor || 1).toString(),
      gamesUsed:      result.gamesUsed || 0,
      confidence:     result.confidence || "low",
      generatedAt:    new Date(),
    }).onConflictDoUpdate({
      target: [ourProjectionsTable.playerId, ourProjectionsTable.statType],
      set: {
        projectedValue: result.projectedValue.toString(),
        weightedAvg:    (result.weightedAvg || result.projectedValue).toStrin
        defenseFactor:  (result.defenseFactor || 1).toString(),
        paceFactor:     (result.paceFactor || 1).toString(),
        gamesUsed:      result.gamesUsed || 0,
        confidence:     result.confidence || "low",
        generatedAt:    new Date(),
      }
    });
    processed++;
  }
} catch (e) {
```

```
      const msg = `${player.fullName} ${line.statType}: ${e instanceof Error ? e.
      errors.push(msg);
      if (errors.length <= 5) console.error("Projection error:", msg);
    }
  }

  if (errors.length > 0) {
    console.warn(`Projection sync: ${errors.length} errors, ${processed} succeede
  }
  return processed;
}
```

9E. COMPLETE buildPlayerIdMappings MASTER FUNCTION

Runs all 4 sport ID mappings in sequence. Call this once on first deploy, then weekly via cron.

```
export async function buildAllPlayerIdMappings(): Promise<void> {
  console.log("Building player ID mappings for all sports...");

  console.log("→ NBA/WNBA...");
  await buildNBAPlayerIdMappings();
  await new Promise(r => setTimeout(r, 1000));

  console.log("→ NHL...");
  await buildNHLPlayerIdMappings();
  await new Promise(r => setTimeout(r, 1000));

  console.log("→ MLB...");
  await buildMLBPlayerIdMappings();
  await new Promise(r => setTimeout(r, 1000));

  console.log("→ NFL...");
  await buildNFLPlayerIdMappings();

  console.log("Player ID mapping complete.");
}
```

Add cron entry in `cron.ts`:

```
// Run weekly on Sunday at 5 AM — catches new players, trades, roster moves
cron.schedule("0 5 * * 0", () => logPull("all-apis", "player-id-mappings", async
  await buildAllPlayerIdMappings();
```

```
  return 1;
}));
```

9F. SURFACE WEATHER IN THE UI

In the Slate Board, add a **weather indicator** column for outdoor MLB and NFL games:

```
// WeatherBadge component
function WeatherBadge({ conditions }: { conditions: any }) {
  if (!conditions || conditions.isIndoor) return null;
  const wind = conditions.windSpeedMph || 0;
  const temp = conditions.tempF || 70;
  const impact = conditions.impact || "neutral";

  if (wind < 10 && temp > 55 && temp < 85) return null; // normal, no badge neede

  const color = impact === "hitter_friendly" ? "success"
    : impact === "pitcher_friendly" ? "danger"
    : "warn";

  const label = wind >= 15
    ? `💨 ${Math.round(wind)}mph ${conditions.windDirectionLabel}`
    : temp < 50 ? `🥶 ${Math.round(temp)}°F`
    : `🌡️ ${Math.round(temp)}°F`;

  return <Pill color={color} size="xs">{label}</Pill>;
}
```

In the prop detail sheet, show the full weather breakdown:

```
GAME ENVIRONMENT
Temp: 61°F | Wind: 18mph NE | Impact: Pitcher-friendly
"Strong wind blowing in from right field. Suppresses fly ball distance.
 Total bases overs have lower probability in these conditions."
```

9G. UPDATED ENVIRONMENT VARIABLES (COMPLETE LIST)

```
# Core (existing)
DATABASE_URL=postgresql://...
CLERK_SECRET_KEY=sk_...
```

```
CLERK_PUBLISHABLE_KEY=pk_...
STRIPE_SECRET_KEY=sk_...
ANTHROPIC_API_KEY=sk-ant-...

# Data APIs (all free, no credit card required for basic tiers)
ODDS_API_KEY=           # the-odds-api.com — sign up free, 500 req/month
OPENWEATHER_API_KEY=    # openweathermap.org — sign up free, 1000 req/day

# Base URLs (do not change unless the APIs move)
PP_API_BASE=https://api.prizepicks.com
ODDS_API_BASE=https://api.the-odds-api.com/v4
MLB_STATS_BASE=https://statsapi.mlb.com/api/v1
NHL_API_BASE=https://api.nhle.com/stats/rest/en
NBA_STATS_BASE=https://stats.nba.com/stats
ESPN_NFL_BASE=https://site.api.espn.com/apis/site/v2/sports/football/nfl
OW_BASE=https://api.openweathermap.org/data/2.5
```

9H. UPDATED COMPLETE CRON SCHEDULE

```
// Every 10 min: PP lines
cron.schedule("*/10 * * * *", () => logPull("prizepicks", "pp-lines", syncPpLines

// Every 20 min: External odds (conserve API credits)
cron.schedule("*/20 * * * *", () => logPull("the-odds-api", "external-odds", sync

// Every hour: Weather update for today's games
cron.schedule("0 * * * *", () => logPull("openweather", "weather", enrichGameWith

// Every hour: Loss limit check
cron.schedule("5 * * * *", checkDailyLossLimits);

// 6 AM daily: Full projection sync (all sports)
cron.schedule("0 6 * * *", () => logPull("all-stats-apis", "projections", syncPro

// 7 AM daily: MLB probable pitchers update (pitchers announced by 7 AM most days
cron.schedule("0 7 * * *", () => logPull("mlb-stats", "mlb-pitchers", async () =>
  // Re-run MLB projections after pitchers are confirmed
  const mlbLines = await db.select({ line: ppLinesTable, player: playersTable })
    .from(ppLinesTable)
    .innerJoin(playersTable, eq(ppLinesTable.playerId, playersTable.id))
    .where(and(eq(ppLinesTable.isActive, true), eq(playersTable.sport, "MLB")));
  let count = 0;
  for (const { line, player } of mlbLines) {
    const result = await buildMLBProjection(player.id, line.statType, player.team
```

```
    if (result) count++;
    await new Promise(r => setTimeout(r, 300));
  }
  return count;
}));

// 5 AM Sunday: Rebuild all player ID mappings (catches trades, rookies)
cron.schedule("0 5 * * 0", () => logPull("all-apis", "player-id-mappings", async
  await buildAllPlayerIdMappings();
  return 1;
}));
```

9I. UPDATED TESTING CHECKLIST (PROJECTIONS)

Add these to the testing checklist in Part 5:

```
PROJECTIONS
[ ] buildAllPlayerIdMappings() runs without error on first deploy
[ ] NBA projection returns value + confidence for active NBA players
[ ] MLB projection: hits different for same batter vs. ace pitcher vs. bad pitche
[ ] MLB projection: Coors Field batter projects higher than Petco Park batter at
[ ] NHL projection returns value for players with active NHL mapping
[ ] NFL projection: confirm ESPN API returns game log data (may need header tweak
[ ] Weather: outdoor MLB games show wind/temp badge in Slate Board
[ ] Weather: roofed stadiums (TOR, TBR, MIA) show no weather badge
[ ] OUR PROJ column shows in Slate Board with confidence badge
[ ] Confidence "low" shown in amber, "medium" in cyan, "high" in green
[ ] Re-running projections at 7 AM updates MLB values after pitchers confirmed
```

UPDATED COST BREAKDOWN

Service	Tier	Cost
Replit	Pro	$25/mo
PostgreSQL	Neon free	$0/mo
The Odds API	Free (500 req/mo)	$0/mo
OpenWeatherMap	Free (1000/day)	$0/mo

NBA Stats API	Free	$0/mo
NHL API	Free	$0/mo
MLB Stats API	Free	$0/mo
ESPN NFL API	Free (unofficial)	$0/mo
Anthropic Claude	~50 sessions/mo	$5–10/mo
Clerk Auth	Free (1 user)	$0/mo
Total		**$30–35/mo**

All projection data is free. No paid sports data service required.